

Enterprise AI-Powered Hotel Management System

Project Overview

Build a complete production-ready AI-powered Hotel Management System with fully separated frontend and backend architecture using modern enterprise technologies.

The platform must support:

- Multi-role authentication
- Hotel room booking
- Real-time room availability
- Restaurant ordering
- Room service requests
- Online payment gateway
- AI-powered recommendations
- AI chatbot assistant
- Housekeeping management
- Analytics dashboard
- Reviews & ratings
- Notifications
- Dynamic pricing
- Staff management
- Inventory tracking
- Invoice generation
- Email notifications
- SMS notification architecture ready
- Firebase integration
- Deployment-ready architecture

The application UI must look premium, modern, elegant, luxurious, responsive, smooth animated, and inspired by top luxury hotel websites.

Use:

- Soft luxury gradients
- Glassmorphism cards
- Elegant typography
- Smooth animations
- Premium spacing
- Professional dashboard UI
- Beautiful booking flow
- Mobile-first responsive design

Avoid:

- Pure dark UI
- Harsh solid colors
- Basic admin template look

Use premium colors:

- Ivory
- Champagne gold
- Royal beige
- Soft teal
- Pearl white
- Elegant navy accents
- Light luxury gradients

EXACT PROJECT STRUCTURE

```
hotel-management-system/  
|  
├── frontend/  
│   ├── public/  
│   ├── src/  
│   │   ├── assets/  
│   │   │   ├── images/  
│   │   │   ├── icons/  
│   │   │   └── styles/  
│   │   ├── components/  
│   │   │   ├── common/  
│   │   │   ├── navbar/  
│   │   │   ├── footer/  
│   │   │   ├── booking/  
│   │   │   ├── rooms/  
│   │   │   ├── dashboard/  
│   │   │   ├── reviews/  
│   │   │   └── forms/  
│   │   └── pages/  
│   │       ├── Home/  
│   │       ├── About/  
│   │       ├── Rooms/  
│   │       ├── Booking/  
│   │       └── Contact/
```

```

├── Login/
├── Register/
├── CustomerDashboard/
├── AdminDashboard/
├── StaffDashboard/
├── Restaurant/
├── Reviews/
├── RoomService/
├── routes/
│   └── AppRoutes.js
├── context/
│   ├── AuthContext.js
│   └── BookingContext.js
├── hooks/
│   ├── useAuth.js
│   └── useBooking.js
├── services/
│   ├── firebase.js
│   ├── authService.js
│   ├── bookingService.js
│   ├── paymentService.js
│   ├── roomService.js
│   └── notificationService.js
├── utils/
│   ├── constants.js
│   ├── helpers.js
│   └── validators.js
├── layouts/
│   ├── MainLayout.js
│   ├── DashboardLayout.js
│   └── AuthLayout.js
├── App.js
├── main.js
├── index.css
├── package.json
├── vite.config.js
├── .env
└── README.md

```

```
|
|
|— backend/
|   |
|   |— config/
|   |   |— firebaseAdmin.js
|   |   └─ db.js
|   |
|   |— controllers/
|   |   |— authController.js
|   |   |— bookingController.js
|   |   |— roomController.js
|   |   |— paymentController.js
|   |   |— customerController.js
|   |   |— reviewController.js
|   |   |— hotelController.js
|   |   └─ staffController.js
|   |
|   |— models/
|   |   |— Booking.js
|   |   |— Room.js
|   |   |— Customer.js
|   |   |— Payment.js
|   |   |— Hotel.js
|   |   |— Staff.js
|   |   └─ Review.js
|   |
|   |— routes/
|   |   |— authRoutes.js
|   |   |— bookingRoutes.js
|   |   |— roomRoutes.js
|   |   |— paymentRoutes.js
|   |   |— customerRoutes.js
|   |   |— reviewRoutes.js
|   |   |— hotelRoutes.js
|   |   └─ staffRoutes.js
|   |
|   |— middleware/
|   |   |— authMiddleware.js
|   |   |— errorMiddleware.js
|   |   └─ roleMiddleware.js
|   |
|   |— services/
|   |   |— paymentService.js
|   |   |— notificationService.js
|   |   └─ bookingService.js
|   |
|   └─ utils/
```

```
| | | generateInvoice.js
| | | calculateGST.js
| | | └─ validators.js
| |
| | └─ uploads/
| |   | hotel-images/
| |   | room-images/
| |   | └─ documents/
| |
| | └─ app.js
| | └─ server.js
| | └─ package.json
| | └─ .env
| | └─ README.md
|
| └─ firebase/
|   | firestore.rules
|   | firebase.json
|   | └─ storage.rules
|
| └─ docs/
|   | API_DOCUMENTATION.md
|   | DATABASE_SCHEMA.md
|   | └─ DEPLOYMENT_GUIDE.md
|
| └─ .gitignore
| └─ README.md
| └─ package.json
```

FRONTEND TECH STACK

Use:

- React.js + Vite
- Tailwind CSS
- React Router DOM
- Axios
- Firebase SDK
- Framer Motion
- React Icons
- Chart.js
- React Hook Form
- Yup Validation
- Zustand or Context API

- React Hot Toast
 - Headless UI
 - Swiper.js
 - Date-fns
 - Recharts
-

BACKEND TECH STACK

Use:

- Node.js
 - Express.js
 - Firebase Admin SDK
 - JWT Authentication
 - Multer File Upload
 - Razorpay SDK
 - Nodemailer
 - Helmet
 - CORS
 - Morgan
 - Compression
 - Express Rate Limit
 - dotenv
 - UUID
 - PDFKit
 - Winston Logger
-

AI FEATURES TO ADD

AI Smart Room Recommendation

Recommend rooms based on:

- Budget
- Family size
- Previous bookings
- Ratings
- Preferred amenities
- Seasonal demand

AI Chatbot Assistant

Capabilities:

- Booking support
- Room suggestions
- FAQ handling
- Restaurant recommendations
- Upselling premium rooms
- Support tickets

AI Dynamic Pricing

Adjust room prices based on:

- Peak season
- Demand
- Occupancy
- Holidays
- Local events

AI Sentiment Analysis

Analyze:

- Reviews
- Feedback
- Ratings
- Complaints

Generate:

- Customer satisfaction score
- Hotel reputation score

AI Analytics Dashboard

Predict:

- Revenue trends
 - Booking trends
 - Occupancy forecasting
 - Cancellation patterns
-

AUTHENTICATION SYSTEM

Implement:

Roles

- Customer
- Admin
- Staff
- Housekeeping
- Restaurant Staff

Authentication Features

- Firebase Authentication
 - JWT token authentication
 - Role-based access control
 - Email verification
 - Forgot password
 - OTP ready architecture
 - Social login support
 - Session management
 - Refresh token handling
-

FRONTEND PAGES

Home Page

Include:

- Luxury hero section
- Animated booking form
- Featured rooms
- Hotel highlights
- Testimonials
- Restaurant showcase
- Gallery slider
- AI recommendation banner
- Newsletter section
- Footer

Rooms Page

Features:

- Room filtering
- Search functionality
- Price sorting
- Availability checking
- Room comparison
- Amenities display
- Dynamic image gallery

Booking Page

Features:

- Multi-step booking
- Date selection
- Guest selection
- Real-time pricing
- Coupon system
- Tax calculation
- Razorpay payment integration

Customer Dashboard

Features:

- Booking history
- Upcoming stays
- Invoice downloads
- Room service requests
- Notifications
- Loyalty points
- Profile settings

Admin Dashboard

Features:

- Revenue charts
- Booking analytics
- Occupancy analytics
- Room management
- Staff management
- Review management

- Complaint center
- AI insights
- Hotel settings

Staff Dashboard

Features:

- Assigned tasks
 - Housekeeping requests
 - Room status updates
 - Restaurant order tracking
 - Notifications
-

RESPONSIVE DESIGN REQUIREMENTS

Must support:

- Mobile
- Tablet
- Laptop
- 4K screens

Use:

- Tailwind responsive utilities
 - Smooth animations
 - Skeleton loaders
 - Lazy loading
 - Code splitting
 - Optimized images
-

FIREBASE SETUP

Firestore Collections

Create:

```
users  
rooms  
bookings  
payments
```

```
reviews
notifications
staff
restaurants
roomServices
housekeeping
analytics
coupons
settings
```

Firestore Rules

Implement:

- Role-based security
- Auth validation
- Upload protection
- Admin-only collections
- Customer ownership validation

RAZORPAY INTEGRATION

Implement:

- Payment order creation
- Payment verification
- Refund handling
- Invoice generation
- GST calculation
- Payment history
- Failed payment recovery

NOTIFICATION SYSTEM

Use:

- Email notifications
- In-app notifications
- Firebase push notifications
- Booking confirmations
- Payment alerts
- Check-in reminders

- Review reminders
-

API REQUIREMENTS

Create REST APIs for:

Auth APIs

- Register
- Login
- Logout
- Refresh token
- Forgot password
- Reset password

Booking APIs

- Create booking
- Update booking
- Cancel booking
- Booking history
- Room availability

Room APIs

- Add room
- Update room
- Delete room
- Upload images
- Room filtering

Review APIs

- Add review
- Update review
- Delete review
- Review analytics

Payment APIs

- Create payment
- Verify payment
- Refund payment
- Download invoice

SECURITY IMPLEMENTATION

Add:

- JWT validation
- Rate limiting
- Helmet security
- CORS configuration
- Input sanitization
- XSS protection
- Secure headers
- Password hashing
- API validation

DASHBOARD UI DESIGN

Use:

- Analytics cards
- Charts
- KPI widgets
- Real-time stats
- Animated graphs
- Booking heatmaps
- Occupancy charts
- Revenue insights

Use Chart.js or Recharts.

ADVANCED FEATURES

Loyalty Program

- Reward points
- Referral bonuses
- Membership tiers
- VIP customers

Smart Check-In

- QR code check-in

- Digital room keys
- ID verification

Inventory Management

Manage:

- Restaurant inventory
- Cleaning inventory
- Consumables

Multi-Hotel Support

Support:

- Hotel chains
- Multiple branches
- Centralized admin

Multi-Language Support

Add:

- English
- Hindi
- Telugu
- Arabic-ready architecture

Multi-Currency Support

Support:

- INR
- USD
- EUR

ENVIRONMENT VARIABLES

Frontend .env

```
VITE_API_BASE_URL=  
VITE_FIREBASE_API_KEY=  
VITE_FIREBASE_AUTH_DOMAIN=
```

```
VITE_FIREBASE_PROJECT_ID=  
VITE_FIREBASE_STORAGE_BUCKET=  
VITE_FIREBASE_MESSAGING_SENDER_ID=  
VITE_FIREBASE_APP_ID=  
VITE_RAZORPAY_KEY_ID=
```

Backend .env

```
PORT=  
JWT_SECRET=  
JWT_EXPIRES_IN=  
FIREBASE_PROJECT_ID=  
FIREBASE_CLIENT_EMAIL=  
FIREBASE_PRIVATE_KEY=  
RAZORPAY_KEY_ID=  
RAZORPAY_KEY_SECRET=  
EMAIL_USER=  
EMAIL_PASS=  
CLIENT_URL=
```

PACKAGE.JSON REQUIREMENTS

Generate complete package.json files for:

- Root
- Frontend
- Backend

Include:

- Scripts
- Dependencies
- Dev dependencies
- Build scripts
- Deployment scripts

DEPLOYMENT REQUIREMENTS

Frontend deploy options:

- Vercel

- Netlify
- Firebase Hosting

Backend deploy options:

- Render
- Railway
- VPS deployment
- Hostinger VPS

Provide:

- Production environment setup
 - Build commands
 - Deployment commands
 - CI/CD ready scripts
-

README REQUIREMENTS

Generate:

Root README

Include:

- Project overview
- Features
- Screenshots section
- Installation guide
- Deployment guide
- Folder structure

Frontend README

Include:

- Frontend setup
- Environment variables
- Build process
- Deployment

Backend README

Include:

- API setup
 - Firebase setup
 - Razorpay setup
 - Email setup
-

CODING STANDARDS

Use:

- Clean architecture
 - Reusable components
 - Service-based structure
 - Custom hooks
 - Proper error handling
 - Loading states
 - Toast notifications
 - API interceptors
 - Environment-based configs
 - Modular structure
 - SOLID principles
-

UI/UX REQUIREMENTS

Create:

- Ultra-modern UI
 - Smooth page transitions
 - Elegant hover effects
 - Animated cards
 - Interactive booking flow
 - Professional tables
 - Premium dashboards
 - Beautiful charts
 - Elegant forms
 - Luxury hotel branding
-

PERFORMANCE OPTIMIZATION

Implement:

- Lazy loading
 - Image optimization
 - Code splitting
 - API caching
 - Memoization
 - Virtualized lists
 - Debounced searches
-

TESTING SETUP

Add:

- Unit testing architecture
 - API testing setup
 - Frontend testing setup
 - Postman collection
-

FINAL OUTPUT REQUIREMENTS

Generate complete:

1. Full frontend code
2. Full backend code
3. Firebase configuration
4. Firestore rules
5. API documentation
6. Database schema
7. Deployment guide
8. Environment variables
9. Razorpay integration
10. Responsive UI
11. Admin dashboard
12. Staff dashboard
13. Customer dashboard
14. Reusable components
15. Authentication system
16. Protected routes
17. AI-powered features

18. Notification system
19. Payment gateway
20. Production-ready architecture

The generated code must be:

- Enterprise-grade
- Fully scalable
- Production-ready
- Secure
- Responsive
- Modern
- Maintainable
- Modular
- High-performance
- Professional UI quality
- Similar quality to premium SaaS products

Generate every file with complete implementation and proper comments.